Do Target Wake Time and Uplink OFDMA Combine? An Offline Reinforcement Learning Study of Joint Scheduling in IEEE 802.11ax

Ahmed Selim Anwar, Ahmed Maksud, Joarder Jafor Sadique,
Charles Wiame, and Marcelo Menezes De Carvalho
Department of Electrical Engineering, *Texas State University*, San Marcos, TX, USA
Emails: {ahmed.selim.anwar, poy18, jjsadique, cwiame, mmcarvalho}@txstate.edu

***Abstract*—IEEE 802.11ax introduces Target Wake Time (TWT) and uplink OFDMA to reduce station energy in dense Wi-Fi deployments. The two mechanisms are usually configured separately, and how they interact when an access point controls both is not established. The difficulty is that a controller must adapt to changing traffic, yet online learning is impractical in a production network, and a hand-tuned rule needs an expert and is unlikely to be optimal. Offline reinforcement learning offers a middle path: a policy is trained from logged data alone, with no exploration on the live network. We cast joint control as a 760-way action, selecting a TWT schedule, a station-to-group assignment, and whether uplink OFDMA is enabled at each decision, and train five offline algorithms on 100,235 logged transitions in ns-3.44. Implicit Q-Learning reaches 107.3 J against a tuned hand-designed controller's 166.5 J, a $35.6\,\%$ reduction at higher throughput, and does so consistently: the spread over ten training runs is 2.6 J. Four of the five methods lose to that controller, and the energy of all five is ordered by how often each enables uplink OFDMA.**

***Index Terms*—IEEE 802.11ax, Target Wake Time, uplink OFDMA, offline reinforcement learning, energy efficiency, conservative Q-learning, off-policy evaluation.**

I. Introduction

Wi-Fi now carries a large share of indoor traffic, and much of it originates from battery-powered devices. IEEE 802.11ax addresses their energy cost with two mechanisms. Target Wake Time (TWT) lets an access point schedule when each station wakes, so radios sleep between service periods [1], [2]. Uplink OFDMA replaces contention with scheduled resource units, so several stations transmit at once [3], [4]. Both reduce energy, and recent work treats their configuration as a scheduling problem in its own right [5], [6].

The two mechanisms are normally configured in isolation. TWT studies fix the OFDMA setting, and OFDMA studies fix or ignore the sleep schedule [7]–[9]. This leaves the joint setting unexamined, which matters because the mechanisms compete. A trigger frame is only useful to a station that is awake to receive it, so every OFDMA round draws on the airtime and the radio states that TWT is trying to conserve.

A single operating point also cannot serve a real deployment. Modern basic service sets carry mixed traffic—latency-sensitive video, periodic telemetry and voice in one cell [10], [11]—and the best schedule under light telemetry is wasteful under a video burst. A controller must adapt, which raises how it should be obtained.

Two obvious answers sit at opposite extremes. A hand-coded rule is cheap to run but needs an expert to tune, and it is unlikely to be optimal away from the conditions it was tuned for. An idealised online learner could in principle track the optimum instant by instant, but it must explore on the live network, and exploration costs latency and risks service degradation [12], [13]. Neither extreme is satisfactory for a deployed access point.

Offline reinforcement learning occupies the space between them [14]. A policy is trained from an already-collected dataset and never acts on the network while learning, so it needs neither exploration nor an expert to label the right action. The price is distribution shift, for which conservative value estimation is the standard remedy [15], [16]. Its wireless applications concentrate on cellular resource management [13], [17], leaving Wi-Fi power save largely unexamined.

We apply it to joint TWT and OFDMA control. Once per decision window the access point selects a TWT schedule, a station-to-group assignment, and whether uplink OFDMA is enabled—a 760-way discrete action. Making the OFDMA bit part of the action, rather than a property of the run, is what makes the problem joint: the policy can enable the scheduler when it pays and disable it when it does not. Five offline algorithms are trained on $100{,}235$ transitions logged in ns-3.44 and compared against hand-designed and non-adaptive baselines.

Our contributions are the following.[1]

- A learned policy that cuts station energy by $35.6\,\%$ against a tuned hand-designed controller at higher throughput, obtained with no online interaction and no expert to set its thresholds. It holds across ten independent

[1]Code, the ns-3 scenario and the frozen results that generate every table and figure: https://github.com/ahmedmaksud/802.11ax-OfflineRL-TWT-OFDMA-WCNC27

training runs, and separately on the seeds whose offered load steps mid-episode and on those whose load is constant.

- Evidence that uplink OFDMA and TWT act as substitutes rather than complements at this operating point: enabling OFDMA raises energy at every TWT configuration tested, and five independently trained policies order by energy exactly as they order by how often each enables it.
- A service-period-aligned trigger scheduler that solicits a station only while its service period can still contain the whole exchange. It removes a race in which a solicited station sleeps before replying, leaving the access point waiting on a radio that has switched off, and halves the measured cost of uplink OFDMA.

II. Related Work

TWT scheduling. TWT has been studied for IoT power saving [1], category-aware grouping [2], latency-sensitive traffic [10]; recent work treats the schedule itself as the object of optimisation [5]. Karaca [7] optimises the TWT mechanism jointly with scheduling, and Roy et al. [18] maximise timely throughput under TWT, allocating OFDMA resources *within* an awake group. These fix a policy by analysis or optimisation; we fit one to logged data.

Uplink OFDMA scheduling. A separate line schedules OFDMA without TWT: deep RL for the OFDMA MAC [3], coexisting latency classes [11], hierarchical RL for joint uplink OFDMA and MU-MIMO [8], multi-agent RL for uplink OFDMA [4], [6], and scalable polling [9]. Objectives are throughput or delay, and station sleep is not part of the control problem. Both mechanisms do appear together [7], [18]; what is absent is a station-energy account in which the OFDMA decision is controlled per decision rather than assumed available, and in which sleeping more competes in the same action space.

Machine learning for Wi-Fi and radio resource management. Online RL has been applied to contention-window control [12] and to the problems above, with exploration on a live network the obstacle to deployment. Offline RL [14] removes it and pays in distribution shift, addressed by conservative value estimation [15], [16]. Wireless applications concentrate on cellular resource management [13], [17], [19], where action spaces are modest; Wi-Fi power save is largely unexamined. We add one observation: at this action cardinality, importance-weighted off-policy evaluation yields too few effective samples to select a checkpoint, so we rank on fitted Q evaluation [20] and validate the ranking.

Relation to our own prior work. Maksud and Carvalho [21], coauthors here, apply online deep RL to dynamic TWT scheduling and share the ns-3 scenario and energy accounting with this paper. Three things differ: the action adds the per-decision OFDMA bit and the group assignment, giving 760 joint actions; learning is offline, with no simulator in the loop; and the question is whether a learned policy beats a tuned hand-designed controller on station energy.

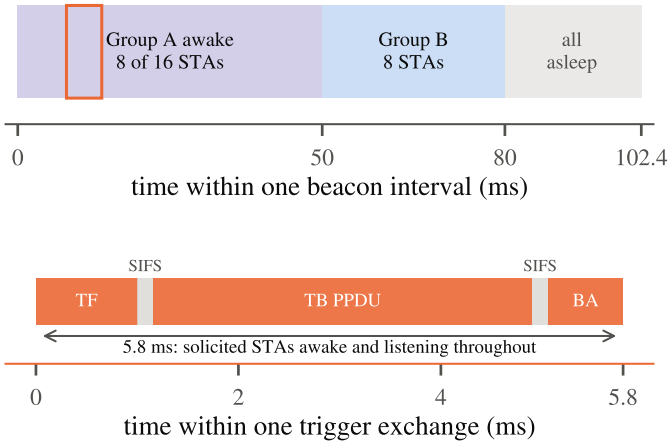


Fig. 1. The control problem. *Top:* one beacon interval under a two-group TWT schedule. *Bottom:* a trigger exchange, drawn to its own scale. Every station a trigger names stays awake and listening for the whole 5.8 ms, which is what makes uplink OFDMA compete with the schedule.

III. System Model

A. Network and traffic

We consider a single basic service set: one access point and $N = 16$ stations on a 20 MHz 802.11ax channel, with a beacon interval of $T_{\mathrm{BI}} = 102.4$ ms. All traffic is uplink.

Stations belong to four device classes, four stations each, chosen to represent the mixed traffic of a real deployment: a low-rate IoT sensor, a video camera, a voice assistant and a video streamer. Their offered IP-layer loads are 0.058, 2.037, 0.030 and 3.400 Mb/s respectively, giving 22.1 Mb/s in aggregate. The access point acts once every $\Delta = 20$ beacon intervals, or 2.048 s, giving 42 decisions per episode.

B. Target Wake Time

A TWT schedule partitions the beacon interval into K service periods of durations $\{w_1, \ldots, w_K\}$. Each station is assigned to one group, and group k is awake from $\sum_{j<k} w_j$ to $\sum_{j\le k} w_j$ within every beacon interval and sleeps outside it. Staggering groups means fewer stations contend at once, at the cost of a shorter window each. We use a catalogue of 20 schedules spanning one to four service periods and 20–90 ms of total wake time, with 19 rules mapping the 16 stations to groups.

C. Uplink OFDMA

When uplink OFDMA is enabled, the access point sends a trigger frame naming a set of stations and the resource unit each may use; those stations reply simultaneously in a trigger-based PPDU. Contention is replaced by scheduling, at a cost: a solicited station must be awake for the trigger and stay awake through its reply and the acknowledgement.

This is where the mechanisms meet: a station outside its service period cannot be solicited, so uplink OFDMA multiplexes only *within* the awake group, and the exchange consumes part of the window the schedule was keeping short.

The scheduler solicits at most four stations per trigger, ns-3's default, while a 20 MHz channel admits nine 26-tone resource units; Section VI reports both limits.

D. Control problem

At decision t the access point observes a state s_t and selects

$$a_t = b_t \cdot 380 + s_t^{\text{sched}} \cdot 19 + g_t, \tag{1}$$

with $s^{\text{sched}} \in \{0, \dots, 19\}$ a schedule, $g \in \{0, \dots, 18\}$ an assignment and $b \in \{0, 1\}$ enabling uplink OFDMA for that window: 760 joint actions. The objective is to minimise station energy without reducing delivered throughput.

IV. Methodology

A. State and reward

The state is a 130-dimensional vector: two aggregate terms, then eight features per station. Per station these are throughput, a normalised demand deficit, three scaled backlog indicators, energy efficiency, drops and energy drawn. All are deltas over the preceding decision window. The state carries no clock, so a policy cannot infer the schedule from time alone.

The reward combines a per-station term r_{sum}, Jain fairness r_{fair}, and a network term r_{net} containing an energy penalty. We use

$$r = 0.30\, z\left(-\tanh \frac{E}{E^*}\right) + 0.55\, z(r_{\text{sum}}) + 0.15\, z(r_{\text{fair}}), \tag{2}$$

where E is the per-decision energy summed over stations, $E^* = 8000$ mJ, and $z(\cdot)$ is standardised on the training split alone. The energy scale matters: the formulation we inherited used $E_0 = 400$ mJ, which saturates the $\tanh$ across the whole operating range and leaves the reward insensitive to energy. Section VI quantifies what that correction is worth.

B. Dataset and training

We log 2,404 episodes, 100,235 transitions (Alg. 1). The ten-decision hold is deliberate: with a fresh draw every decision, measured energy is flat against schedule tightness, and only a sustained schedule makes the gradient visible. A uniform behaviour policy is also the conservative choice. It is the hardest case for offline RL, because the data contains no competent trajectory to imitate—which is why behaviour cloning finishes last here—so a deployment logging from a working controller would start from strictly better coverage of the actions worth taking. Splits are by episode, never by transition—2,043 training, 361 held out—and we train behaviour cloning, DQN+CQL, Double DQN, dueling DQN+CQL and IQL [15], [16], [22] at ten initialisations each.

C. Checkpoint selection

Selecting a checkpoint without touching the network is itself a problem. Importance-sampling estimates are degenerate here: for a deterministic policy against a behaviour policy uniform over A actions, the effective sample size reduces to the state-action matches—a few hundred out of 15,025 held-out transitions. We therefore rank by fitted Q evaluation [20], fitting a critic on the training split and evaluating at held-out episode starts, and validate that ranking against the simulator rather than assume it sound.

Algorithm 1 Offline joint TWT/OFDMA control

1: **Collect** $\mathcal{D}$: for each episode, draw $a \sim \text{Unif}(760)$, hold it for $H = 10$ decisions, log (s, a, r, s')
2: **Split** $\mathcal{D}$ by episode into $\mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{ho}}$
3: **for** each algorithm m and seed $\sigma \in \{42..51\}$ **do**
4: $\quad\theta_{m,\sigma} \leftarrow$ train m on $\mathcal{D}_{\text{tr}}$, snapshot every 50 epochs
5: $\quad k^\star \leftarrow \arg\max_k \text{FQE}(\theta^{(k)}_{m,\sigma}; \mathcal{D}_{\text{tr}}, \mathcal{D}_{\text{ho}})$ {no simulator}
6: **end for**
7: **Deploy** $\theta^{(k^\star)}$: each decision, take $a_t = \arg\max_a Q(s_t, a)$, decode $(b_t, s_t^{\text{sched}}, g_t)$, and if b_t solicit only stations whose service period has ≥ 5.8 ms left

D. Service-period-aligned triggering

A trigger exchange started near the end of a service period cannot finish inside it. The station sleeps before it replies, and the access point is left awaiting a reply from a radio that has switched off. That is a protocol fault, not a missed transmission opportunity.

We therefore solicit a station only while its service period still has room for the entire exchange, which we take as 5.8 ms, covering the trigger, two short interframe spaces, a maximum-duration trigger-based PPDU and the acknowledgement. This aligns uplink OFDMA with the awake window by construction.

V. Experimental Setup

A. Simulator and parameters

All experiments use ns-3.44 with the ns3-ai interface, which lets a Python policy act in a running simulation. Table I lists every parameter, naming the sweep wherever one chose the value.

B. Seeds and protocol

Three disjoint seed sets are used, so no policy is evaluated on data that informed it: collection from 30000, tuning on 80000–80014, and all reported results on 90000–90049.

The fifty evaluation seeds are *paired*: a seed fixes the traffic, the time of any load change and its multiplier, so every policy meets an identical environment. Pairing removes environment variance, which is what lets fifty seeds resolve differences that would otherwise need many more. Energy is reported over the 42-decision control window, excluding the warm-up.

C. Baselines

Nine non-learned scenarios span the configuration space, from no power save to a tuned hand-designed controller; Table II lists them.

That controller is the comparator that matters: a delay-triggered hysteresis rule raising the duty cycle when observed delay crosses an upper threshold and lowering it below a lower one. Its thresholds, schedule pair and OFDMA mode come from a staged coordinate search on the tuning seeds, minimising energy subject to a service floor, so it cannot win by refusing traffic. The static baseline is chosen the same way.

TABLE I
SIMULATION AND TRAINING CONFIGURATION. EVERY VALUE IS THE ONE USED TO PRODUCE THE REPORTED RESULTS; WHERE A SWEEP CHOSE IT, THE SWEEP IS NAMED.

Scenario (ns-3.44)		
stations	16	four device classes, four each
channel	20 MHz	802.11ax, single BSS
STAs per trigger	4	ns-3 default; 20 MHz fits 9
CCA-busy current	273 mA	ns-3 default; TX/RX 232/66
beacon interval	102.4 ms	
decision interval Δ	20 BI = 2.048 s	42 decisions per episode
episode length	86 s	
offered load	0.058/2.037/ 0.030/3.400 Mb/s	IoT / camera / voice / streamer; 22.1 Mb/s total
load step	$\times\{1.5, 2.0, 2.5\}$	in 60 % of episodes, at $U[30, 70]$ s; seed-determined
Dataset		
transitions	100,235	2,404 episodes
split	85,210 / 15,025	by episode, 2,043 / 361
behaviour policy	uniform over 760	all 760 actions present
Training (all five algorithms)		
network	130–128–128–760	ReLU; dueling splits the head
optimiser	Adam, lr 10^{-3}	defaults otherwise
batch size	512	149 steps per epoch
epochs	200	milestone every 50
discount γ	0.95	
Bellman target clip	± 50	chosen so $\lvert V\rvert = 23$–35 fits
CQL coefficient α	0.5	from an FQE sweep over $\{0.5, 1.0, 2.0\}$
IQL expectile	0.7	
training seeds	42–51	ten independent fits per algorithm (Table II)
Selection and evaluation		
model selection	FQE, 3×10^4 iters	episode-disjoint holdout
convergence gate	none	see Section VI
evaluation seeds	50 shared	disjoint from training
heuristic tuning seeds	15	80000–80014, disjoint

VI. RESULTS

A. The two mechanisms do not combine

Table II holds three pairs of rows differing only in the OFDMA bit, and in every pair enabling uplink OFDMA costs energy: $+62.8\,\%$ under the static schedule, $+33.6\,\%$ under random TWT, $+8.3\,\%$ with TWT off.

Throughput falls in all three pairs, so this is not a throughput-for-energy trade: the OFDMA arms are worse on both axes.

The ordering is the informative part: the penalty is largest where TWT sleeps hardest and smallest where it does not sleep at all, because soliciting a station competes with the schedule trying to keep it asleep.

B. The cost is carrier sensing, not lost sleep

Our first explanation was that triggers wake sleeping stations. The traces refute it. Mean sleep occupancy—the fraction of the control window a station spends in PHY sleep, averaged over the sixteen—differs by under 0.8 percentage points between the OFDMA arms. Stations sleep just as much; their awake time simply costs more.

The PHY-state traces show where it goes: of the 113.7 J of extra energy, $69\,\%$ is carrier-sense busy and $24\,\%$ receiving, only $9\,\%$ transmission. A solicited station listens through the trigger, the allocation and the acknowledgement, charged at the CCA-busy current.

The scenario inherits ns-3's default CCA-busy current of 273 mA while setting its own receive current to 66 mA, so sensing the medium costs more than transmitting on it—not a realisable radio. We therefore re-evaluated every arm the claims rest on at 66 mA, same policies, same seeds. Energies fall by 21 to $27\,\%$ and not uniformly, but both conclusions hold: OFDMA still costs $+57.8\,\%$ under the static schedule rather than $+62.8\,\%$, and IQL still beats the hand-designed controller, by $31.2\,\%$ (84.6 against 123.1 J, CI $[-32.2, -30.2]\,\%$) rather than $35.6\,\%$. Signs survive the power profile; magnitudes do not.

C. One algorithm beats the tuned rule

Given a free choice of OFDMA mode the tuned controller disables it, worth 99 J (265.5 against 166.5 J) at slightly better throughput. We take the stronger form, 166.5 J at 19.22 Mb/s, as the baseline.

IQL reaches 107.3 ± 2.6 J, the spread being the standard deviation over ten independent training runs rather than a confidence interval: what training once should land on. That is a $35.6\,\%$ reduction, CI $[-36.5, -34.7]\,\%$, $p = 1.8 \times 10^{-15}$ on fifty paired seeds. Nothing is traded for it: throughput rises to 20.76 from 19.22 Mb/s and mean delay falls by 83 ms ($p = 9.8 \times 10^{-9}$ paired), while the change in budget violations is not statistically detectable ($p = 0.07$). Its worst fit of ten, 112.4 J, still beats every other method's best.

Absolute delays warrant a word: the cell carries a standing queue, flat at 2.6–2.9 s across all 42 decisions and every arm, including the one with neither mechanism. It reflects queue depth under this load rather than a response to scheduling, so we read delay as a guard against winning by starving traffic, not as a latency claim.

D. Four of five do not, and selection is why

Table II is the reason we claim this for IQL and not for offline RL as a family. Read the *typical fit* column, which compares the mean over the ten fits and so describes what training once yields. DQN+CQL is $15.9\,\%$ worse than the hand-designed rule ($p = 8.6 \times 10^{-5}$) and dueling DQN+CQL is $16.6\,\%$ worse ($p = 4.3 \times 10^{-6}$). DDQN and behaviour cloning are worse still.

Now read the *best fit* column. There the two CQL variants sit within $1.6\,\%$ of the baseline, and neither difference is statistically detectable: $p = 0.74$ and $p = 0.55$, with the paired difference's confidence interval spanning zero. The gap between the two columns is the whole problem. A practitioner who trains once gets a policy $16\,\%$ worse than a hand-written rule. A practitioner who can identify the best of ten gets a tie. Identifying it requires measuring all ten on the network, which is the cost offline RL exists to avoid.

TABLE II

EVERY SCENARIO ON THE SAME 50 PAIRED EVALUATION SEEDS, ONE SIMULATOR AND ONE 42-DECISION MEASUREMENT WINDOW. WHERE UPLINK OFDMA IS ENABLED THE SCHEDULER IS SERVICE-PERIOD ALIGNED AND THE SLEEP GUARD IS DISABLED, SO NO ROW DEPENDS ON THAT WORKAROUND. THE HAND-DESIGNED CONTROLLER IS TUNED ON DISJOINT SEEDS UNDER A SERVICE FLOOR; THE ROW MARKED *free choice* LETS THAT SEARCH PICK ITS OWN OFDMA MODE, AND IT PICKS OFF. LEARNED ROWS GIVE THE MEAN AND STANDARD DEVIATION OVER 10 TRAINING SEEDS. THE LAST THREE COLUMNS COMPARE EACH LEARNED METHOD AGAINST THE *free choice* CONTROLLER AT 166.5 J, WILCOXON SIGNED-RANK: *typical* USES THE PER-SEED MEAN OVER THE TEN FITS, WHICH IS WHAT TRAINING ONCE YIELDS, AND *best* USES THE SINGLE LOWEST-ENERGY FIT, WHICH CAN ONLY BE IDENTIFIED BY MEASURING ALL TEN ON THE NETWORK.

						typical fit		best fit
Scenario	E (J)	tput (Mb/s)	delay (ms)	viol.	OFDMA	Δ vs hand	p	Δ vs hand
No TWT, no OFDMA	441.1	24.95	2637	0.24	0 %			
Static TWT	173.3	21.03	2789	0.37	0 %			
Static TWT + OFDMA	282.2	19.82	2823	0.40	100 %			
Random TWT	171.1	14.47	3159	0.63	0 %			
Random TWT + OFDMA	228.5	13.51	3188	0.66	100 %			
OFDMA only, no TWT	477.8	24.30	2652	0.25	100 %			
Random joint, no TWT	198.8	14.29	3157	0.63	48 %			
Hand-designed, OFDMA on	265.5	18.37	2904	0.43	100 %			
Hand-designed, *free choice*	166.5	19.22	2888	0.41	0 %			
offline RL, 10 training seeds each								
IQL	107.3 ± 2.6	20.76	2805	0.38	0 %	-35.6 %	10^{-15}	-37.2 %
DQN+CQL	193.0 ± 10.1	20.54	2829	0.40	23 %	$+15.9$ %	10^{-5}	$+1.6$ %
Dueling+CQL	194.1 ± 14.7	20.49	2834	0.40	25 %	$+16.6$ %	10^{-6}	$+0.7$ %
DDQN	222.7 ± 13.5	15.64	3076	0.58	43 %	$+33.8$ %	10^{-15}	$+23.7$ %
BC	255.1 ± 12.4	19.78	2875	0.43	48 %	$+53.2$ %	10^{-15}	$+42.7$ %

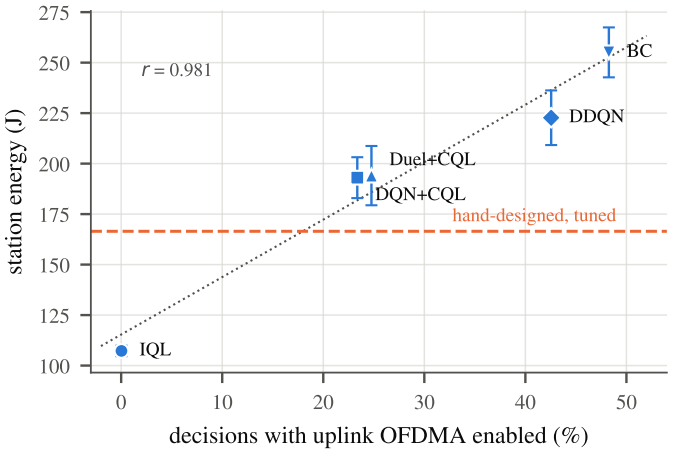


Fig. 2. Station energy against how often each learned policy enables uplink OFDMA; bars are the standard deviation over ten training seeds. The association is almost deterministic ($r = 0.981$), and the method that learned to stop soliciting is the one that beats the tuned controller (dashed).

IQL is the exception because it barely varies: its across-fit standard deviation is 2.6 J against 10.1 to 14.7 J for the others, so which fit you deploy does not matter.

E. What separates the methods is the OFDMA bit

The five methods order by energy exactly as they order by how often they enable the scheduler (Fig. 2): IQL 0 % through behaviour cloning at 48 %—the last because the logging policy chose the OFDMA bit by coin flip, and imitating it reproduces that.

The method that learned to stop soliciting is the one that wins. The coordinate search concluded the same by a different route—a staged grid over thresholds rather than value iteration over 760 actions—so two optimisers of different character agree that at this operating point the mechanisms substitute. The obvious objection is that we crippled the scheduler: ns-3 solicits four stations per trigger by default, while schedule 11 leaves eight awake and the channel admits nine resource units, so half the awake set was unreachable. Raising the limit to nine makes uplink OFDMA worse, not better—+87.4 % against the no-OFDMA arm instead of +62.8 % under the static schedule, and +83.3 % instead of +59.5 % for the hand-designed controller, both +15 % on the default at $p = 2 \times 10^{-15}$ with throughput falling at each step. Solicitation cost scales with the stations a trigger names; the benefit scales with the data they have, and in this mix they have little.

Each seed either holds its offered load constant or steps it by 1.5 to 2.5× partway through. Splitting the fifty on that flag tests whether the policy needs a settled traffic pattern; it does not—35.1 % below the baseline on the 19 constant-load seeds ($p = 3.8 \times 10^{-6}$) and 35.8 % on the 31 stepped ones ($p = 9.3 \times 10^{-10}$), with higher throughput in both.

F. What the measurement rests on

Four instrumentation faults each produced the opposite headline, and all four were invisible in aggregate statistics. A convergence gate admitted only snapshots whose FQE drift fell below 2 %; none passed, so selection fell back to the final epoch, which both FQE and the simulator rank last of four. Evaluating every snapshot in ns-3 gives a Spearman correlation of -1.000 between FQE value and energy: the ranking was sound, the gate discarded it, and it cost 6.5 % energy. We now rank on FQE alone. The inherited reward charged $\tanh(E/E_0)$ at $E_0 = 400$ mJ against an E spanning 1390 to 14,167 mJ, saturated for 99.99 % of transitions; retraining all ten fits on it

costs 13.3 J (120.7 ± 5.6 against 107.3 ± 2.6, $p = 3.6 \times 10^{-15}$ paired and 1.2×10^{-4} across fits), though it still beats the baseline by $27.5\,\%$, so the repair improves the result rather than producing it. Two harnesses measured different windows, one including a warm-up burning $26.6\,\%$ of episode energy in $9.8\,\%$ of it; matched to the 42-decision window they agree to $4.5\,\%$. Finally the scenario deferred each PHY sleep by 8 ms with the OFDMA bit set, worth $+62.0\,\%$ on its own—two thirds of the $+92.5\,\%$ we had charged to the protocol. Sleep occupancy moved by under $0.8\,\%$ between those arms, which is why we first read it as harmless: it changed when stations slept, not how much. Service-period alignment removes the race, and every OFDMA arm here runs without the deferral.

VII. Conclusion

A controller for TWT and uplink OFDMA has to adapt, and neither obvious way of obtaining one is satisfactory: a hand-coded rule needs an expert and is tuned for conditions that change, while an online learner has to explore on a network carrying real users. We tested the middle path. Five offline algorithms were trained on 100,235 logged transitions in ns-3.44, ten initialisations each, selecting a TWT schedule, a group assignment and the uplink OFDMA bit per decision window—760 actions—and compared against nine non-learned scenarios on fifty paired seeds under one simulator and one window.

Two findings follow. First, IQL reduces station energy by $35.6\,\%$ against a hand-designed controller tuned under a service floor, at higher throughput and consistently across ten fits. Second, the two mechanisms substitute rather than combine here: enabling uplink OFDMA raises energy at every TWT setting tested, the five methods order by how often each enables it, and an independent coordinate search disables it when given the choice.

The result is about IQL, not offline RL. Four of five methods lose to the hand-designed rule, and the two CQL variants tie only if their best of ten fits can be identified—which takes the network measurements the method was adopted to avoid. Low across-fit variance is what makes IQL usable, so checkpoint selection, not policy optimisation, is the remaining difficulty.

Three limits bound the claims. The topology is one basic service set, sixteen stations and 20 MHz; we tested solicitation limits of four and nine and the substitution strengthened at both, but we cannot speak beyond them. The station mix and the traffic model are identical across every arm, so the comparisons are internally controlled but not shown to transfer. The energy model also mixes two power profiles, leaving CCA-busy at ns-3's default 273 mA—measured on an 802.11n laptop card [23]—against its own 66 mA receive current. Re-evaluating at 66 mA moves arm energies by 21 to $27\,\%$ and unevenly, yet changes no ordering and neither conclusion (Section VI); since the policies were trained under the shipped profile, that repricing measures the same behaviour under a different energy model, not what a policy would have learned under it. Finally the logged actions were held for ten decisions; deploying at that cadence changes energy by $-1.0\,\%$ ($p = 0.12$), so the mismatch does not appear to bite.

Whether the substitution reverses at the higher station counts, wider channels and larger solicitation sets where uplink OFDMA is expected to pay is the natural next step.

References

[1] Q. Chen, Z. Weng, X. Xu, and G. Chen, "A target wake time scheduling scheme for uplink multiuser transmission in IEEE 802.11ax-based next generation WLANs," *IEEE Access*, vol. 7, pp. 158 207–158 222, 2019.

[2] W. Qiu, G. Chen, K. N. Nguyen, A. Sehgal, P. Nayak, and J. Choi, "Category-based 802.11ax target wake time solution," *IEEE Access*, vol. 9, pp. 100 154–100 172, 2021.

[3] D. Kotagiri, K. Nihei, and T. Li, "Distributed convolutional deep reinforcement learning based OFDMA MAC for 802.11ax," in *IEEE ICC*, 2021, pp. 1–6.

[4] M. Han, X. Sun, W. Zhan, Y. Gao, and Y. Jiang, "Multi-agent reinforcement learning based uplink OFDMA for IEEE 802.11ax networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 8, pp. 8868–8882, 2024.

[5] F. Busacca, S. Mangione, S. Palazzo, F. Restuccia, and I. Tinnirello, "Target wake time scheduling for time-sensitive and energy-efficient Wi-Fi networks," *IEEE Trans. Mobile Comput.*, vol. 25, no. 3, 2026.

[6] Y. Liu *et al.*, "Priority-aware multi-agent reinforcement learning for uplink OFDMA in Wi-Fi networks," *IEEE Internet Things J.*, 2025.

[7] M. Karaca, "Joint optimization of TWT mechanism and scheduling for IEEE 802.11ax," arXiv:2006.02235, 2020.

[8] H. Noh, H. Lee, and H. J. Yang, "Joint optimization on uplink OFDMA and MU-MIMO for IEEE 802.11ax: Deep hierarchical reinforcement learning approach," *IEEE Commun. Lett.*, vol. 28, no. 8, pp. 1800–1804, 2024.

[9] D. D. Agbeve, A. Belogaev, W. Sandra, C. Lylon, and J. Famaey, "A2p: A scalable OFDMA polling algorithm for time-sensitive Wi-Fi networks," arXiv:2502.00430, 2025.

[10] C. Yang, J. Lee, and S. Bahk, "Target wake time scheduling strategies for uplink transmission in IEEE 802.11ax networks," in *IEEE WCNC*, 2021, pp. 1–6.

[11] Q. Tan, J. He, and Y. Gao, "Deep reinforcement learning based OFDMA scheduling for WiFi networks with coexisting latency-sensitive and high-throughput services," in *ICTC*, 2024, pp. 146–150.

[12] W. Wydmański and S. Szott, "Contention window optimization in IEEE 802.11ax networks with deep reinforcement learning," in *IEEE WCNC*, 2021, pp. 1–6.

[13] K. Yang, C. Shi, C. Shen, J. Yang, S.-P. Yeh, and J. J. Sydir, "Offline reinforcement learning for wireless network optimization with mixture datasets," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 12 703–12 716, 2024.

[14] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," arXiv:2005.01643, 2020.

[15] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for offline reinforcement learning," in *NeurIPS*, 2020, pp. 1179–1191.

[16] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-Learning," in *ICLR*, 2022.

[17] E. Eldeeb and H. Alves, "An offline multi-agent reinforcement learning framework for radio resource management," *IEEE Trans. Mobile Comput.*, 2026, arXiv:2501.12991.

[18] R. Roy, R. V. Bhat, P. Hathi, N. Akhtar, and N. M. Balasubramanya, "Maximization of timely throughput with target wake time in IEEE 802.11ax," in *IEEE ICC*, 2023, pp. 647–652.

[19] E. Eldeeb and H. Alves, "Offline and distributional reinforcement learning for radio resource management," in *EuCNC/6G Summit*, 2025, arXiv:2409.16764.

[20] H. M. Le, C. Voloshin, and Y. Yue, "Batch policy learning under constraints," in *ICML*, 2019, pp. 3703–3712.

[21] A. Maksud and M. M. Carvalho, "Deep reinforcement learning-based dynamic TWT scheduling for heterogeneous Wi-Fi networks," in *ICCCN*, 2026.

[22] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *AAAI*, 2016, pp. 2094–2100.

[23] D. Halperin, B. Greenstein, A. Sheth, and D. Wetherall, "Demystifying 802.11n power consumption," in *HotPower'10*, 2010.